操作系统实验

题　　目	:	实验6
专业名称	:	计算机科学与技术
授课教师	:	陈鹏飞
学生姓名	:	陈安康
学　　号	:	22320021
实验地点	:	实验中心D402
日　　期	:	2024/5/16

任务一：

1.代码复现：

在教程中，对于“消失的汉堡”这一问题，教程分别给出了自旋锁与信号量的解决方法。

自旋锁每次只允许一个进程访问操作共享变量（这里是汉堡），直到释放锁之后再由其他进程获得访问权限。当母亲获得汉堡数量的访问权后直到其释放锁之前，儿子无法访问汉堡，只能在其执行时间不断占据CPU自旋，尝试获取锁。信号量初始化资源数目（这里是1），每当有进程进入边界区，便消耗信号量，如果信号量为零则说明没有资源，进程等待阻塞，直到其他进程释放信号量资源后再将其唤醒。

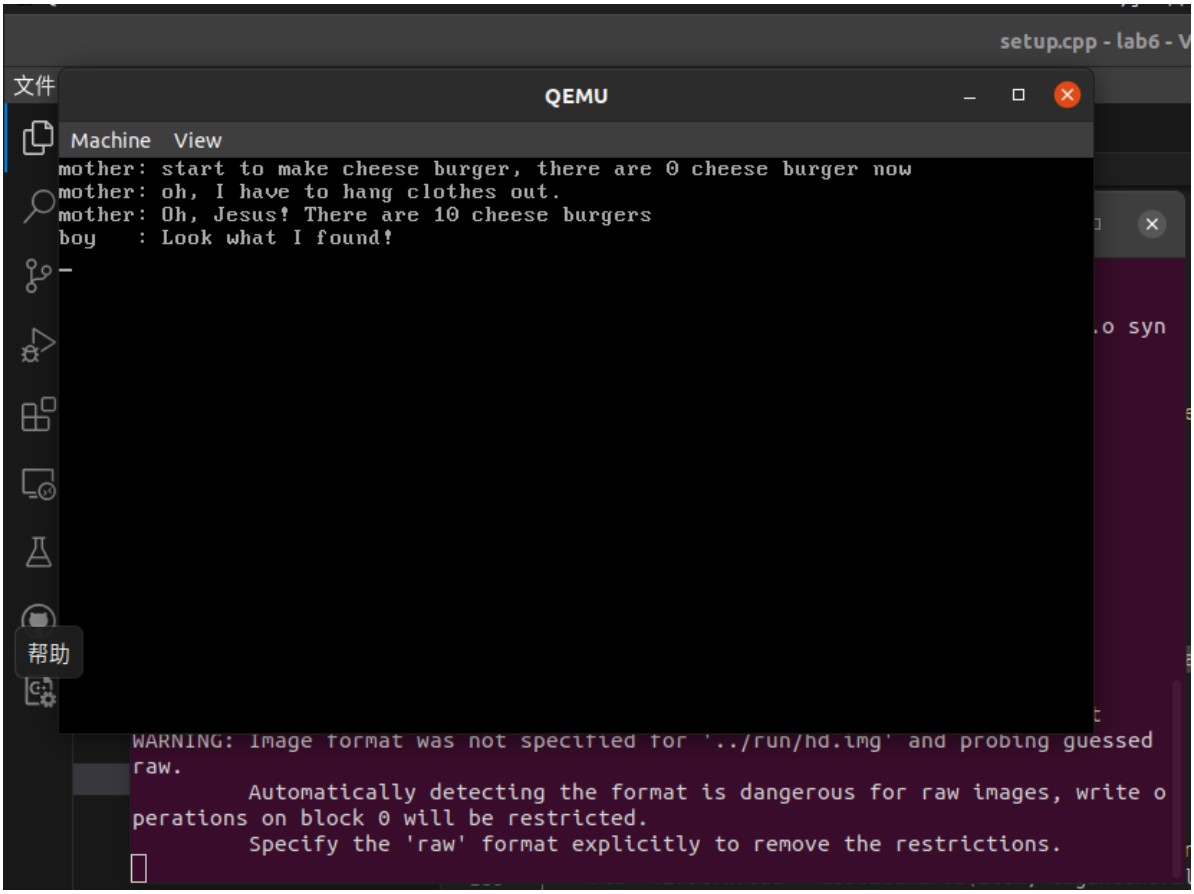
实现上，自旋锁采用xchg原子指令实现，思路是不断尝试将本地变量（设置为1）与锁进行交换来完成判断锁与改变锁两个功能，这里的核心就是xchg操作是原子操作，这样，通过交换操作，可以保证访问锁与设置锁，保存锁的原值（供判断）三个连续操作是原子不可分割的，从而保证对锁的操作是互斥的，防止因为调度导致两个及以上的线程进入边界区造成错误。为了达成这个效果，**要求访问锁的同时一定要更新锁的值**，不然，访问获得锁时没有改变锁，就会出现两个线程同时获得值为0的锁，导致访问错误。

由于xchg指令的特性，不允许操作的是两个内存地址，需要首先获得本地变量的值，放入寄存器，然后再进行交换，如果首先访问共享锁，则没有立即更改其值，会出现两个进程同时访问锁的情况，这时锁为0就会使两个线程同时认为锁没有关闭，进入边界区，导致访问错误，所以首先将本地变量值放入寄存器，通过xchg操作进行交换后就得到了锁的初值以及对锁访问并更新（一系列全部是原子的）。从而实现自旋锁控制边界区的访问。

信号量实现上，创建了一个计数信号量标定资源数量（这里只有cheese_burger一个变量，所以最大资源量为1）、自旋锁（保证对计数信号量操作互斥）和阻塞队列，获取锁的线程判断计数信号量是否为0，如果没有可用资源，则将线程阻塞，调度下一个线程，等到有线程释放资源后，唤起线程，放入等待队列中再次进行判定。这里通过一个队列，设置线程的状态实现线程挂起，因为调度函数只有再线程为运行状态时才会将其放入就绪队列，而通过设置线程状态为BLOCKED，该线程不会放入就绪队列，只会在阻塞队列中，因此实现了线程的阻塞。唤醒时，只需要改变线程状态并将其手动放入就绪队列即可。因为不能保证唤醒队列一定会获得信号量，所以要再次进行判断。

在教程的实现上，并没有将边界区的资源封装进去，而是单独实现锁或信号量，在执行对边界区资源的操作之前加锁，操作完之后释放锁，来完成线程的同步与互斥。

复现教程中自旋锁与信号量的实现方法，实现效果的截图是一样的，这里只放一个，如下：



另一种自旋锁实现：

因为在线程**首次访问锁时，一定要同时更新锁的值（这里是指进行更新操作，不一定真正修改锁的值），也就是说访问锁与更新锁一定要是原子的**，否则会出现多个线程同时进入边界区的情况。假设尝试获得锁的一系列操作中，访问锁的操作是a，更新锁的操作是b，其操作是分离的，那么假设当前锁为0时，有两个线程，分别记为thread1与thread2，则可能存在如下调度顺序：

	thread1	thread2
1	a	
2		a
3	b	
4		b

这样的话，thread1与thread2就会同时访问锁获得值为0，此时以当前值进行判断，两个线程都以为自己获得了锁，同时进入边界区，导致错误。所以一定要在初次访问获得锁时就更新锁的值，教程中的xchg就是本质上保证了这一点（对于访问顺序的约束其实本质也是确保第一次访问共享变量时就更新其值）基于这一点，我选择**使用lock前缀修饰的add操作来实现锁**，思路如下：

获取锁的操作中，设置本地变量flag，用来保存对锁的判断结果，然后设置循环，不断执行操作asm_atomic_lock判断锁的状态，直到锁空闲，获取锁，跳出循环。

对于操作asm_atomic_lock，采用汇编语言实现，传入的变量为锁以及判断变量的地址，在获取锁的地址之后，直接对锁的值进行加一，这样就实现了访问锁与更新锁的操作是原子的，同时将锁的值置为1(这个稍后说明原因)由于加一了，所以我们对锁的判断从判断是否是0变成判断是否是1，因此如果获得的锁的值为1，那么说明锁是空闲的，设置flag值为0跳出循环，否则说明锁被其他线程获取，则一直循环尝试直到获取锁。

```cpp
void SpinLock::my_lock() {
    uint32 flag = 1;
    do {
        asm_atomic_lock(&bolt, &flag);
    } while(flag);
}
```

```cpp
extern "C" int asm_atomic_lock(uint32* lock, uint32* flag);
```

```
asm_atomic_lock:
    push ebp
    mov ebp, esp
    pushad
    ;获得锁之后直接加一
    mov ebx, [ebp + 4 * 2]
    lock add dword[ebx], 1
    mov eax, [ebx]
    mov dword[ebx], 1
    mov ebx, [ebp + 4 * 3]
    cmp eax, 1
    jne label
    mov dword[ebx], 0
    jmp return

label:
    mov dword[ebx], 1
    jmp return

return:
    popad
    pop ebp
    ret
```

这样，由于保证访问更新锁的原子性，那么就能保证线程之间在获取锁时是互斥的。

释放锁同教程的自旋锁操作，直接修改锁为0即可。

为什么访问锁后要对锁进行设置为1？我是这样考虑的：因为如果放任锁的值不断增加，那么可能会出现这样一个情况（这需要十分多的线程，感觉实际上不太可能发生）：**在一个进程获取锁并进入边界区执行操作的期间（一个进程霸占锁之后不知什么原因在边界区不出来了），执行时间太长，有非常多的进程尝试获取锁，锁不断加一，导致超出int类型边界，锁溢出，又变为0，从而导致有的进程进入边界区，从而发生错误。**虽然这种情况十分不常见，但仍然是会存在的，因此在锁加一后，我们将锁复原为1，这样，除非所有的线程都恰巧在加一操作之后，置一操作之前被调度（这种可能性微乎其微，可以认为不可能发生），否则不会出现上述情况，从而保证了正确性。

这个实现依附于教程的实现的类，位于文件夹2中。

用这个锁来解决“消失的汉堡”问题，成功解决：

```
void a_mother(void *arg)
{
    aLock.my_lock();
    int delay = 0;

    printf("mother: start to make cheese burger, there are %d cheese burger now\n", cheese_burger);
    // make 10 cheese_burger
    cheese_burger += 10;

    printf("mother: oh, I have to hang clothes out.\n");
    // hanging clothes out
    delay = 0xfffffff;
    while (delay)
        --delay;
    // done

    printf("mother: Oh, Jesus! There are %d cheese burgers\n", cheese_burger);
    aLock.unlock();
}
```

```
void a_naughty_boy(void *arg)
{
    aLock.my_lock();
    printf("boy    : Look what I found!\n");
    // eat all cheese_burgers out secretly
    cheese_burger -= 10;
    // run away as fast as possible
    aLock.unlock();
}
```

```
void a_mother(void *arg)
{
    aLock.my_lock();
    int delay = 0;

    printf("mother: start to make cheese burge
    // make 10 cheese_burger
    cheese_burger += 10;

    printf("mother: oh, I have to hang clothes
    // hanging clothes out
    delay = 0xfffffff;
    while (delay)
        --delay;
    // done

    printf("mother: Oh, Jesus! There are %d ch
    aLock.unlock();
}

void a_naughty_boy(void *arg)
{
    aLock.my_lock();
    printf("boy    : Look what I found!\n");
    // eat all cheese_burgers out secretly
    cheese_burger -= 10;
    // run away as fast as possible
    aLock.unlock();
}
```

QEMU

Machine View

```
mother: start to make cheese burger, there are 0 cheese burger now
mother: oh, I have to hang clothes out.
mother: Oh, Jesus! There are 10 cheese burgers
boy    : Look what I found!
```

生产者消费者问题：

代码位于文件夹2中。

生产者消费者问题复现：

首先我们要创建一个缓冲区，里面定义了一个字符数组，还有指针head指向队列第一个元素，end指向下一个应当放置元素的位置，以及目前队列的大小current_size，当head == end，通过current_size来判断队列为满还是空。该类还定义了模拟放置数据以及取数据的方法，每次模拟都会对队列合法性进行判断，**如果队列不合法，首先打印错误信息，然后通过触发除零错误来进行中断。**定义如下，定义在setup.cpp中

```
#define BUFFER_SIZE 10
class Buffer {
    //缓冲区，大小定义为10个字节
    char buffer[BUFFER_SIZE];
    //head是指向队列的第一个数据，也是消费者开始拿数据的第一个位置
    //end指向队尾的下一个元素，也就是生产者需要放置数据的位置
    //当head == end，current_size = 0，缓冲区为空，current_size = BUFFER_SIZE,缓冲区满
    int current_size;
    int head, end;
public:
    Buffer() : current_size(0), head(0), end(0) {

    }
    void put_data(char a) {
        buffer[end] = a;
        end = (end + 1) % BUFFER_SIZE;
        ++current_size;
        printf("producer has put a data %c in buffer, the current_size of buffer is %d\n", a, current_size);
        isvaild();
    }
    void get_data() {
        head = (head + 1) % BUFFER_SIZE;
        --current_size;
        printf("producer has get a data in buffer, the current_size of buffer is %d\n", current_size);
        isvaild();
    }
    void isvaild() {
        if(current_size > BUFFER_SIZE || current_size < 0) {
            printf("the buffer has go wrong!!!");
            // 触发除0错误来中断执行
            int interrupt = 1 / 0;
        }
    }

};
```

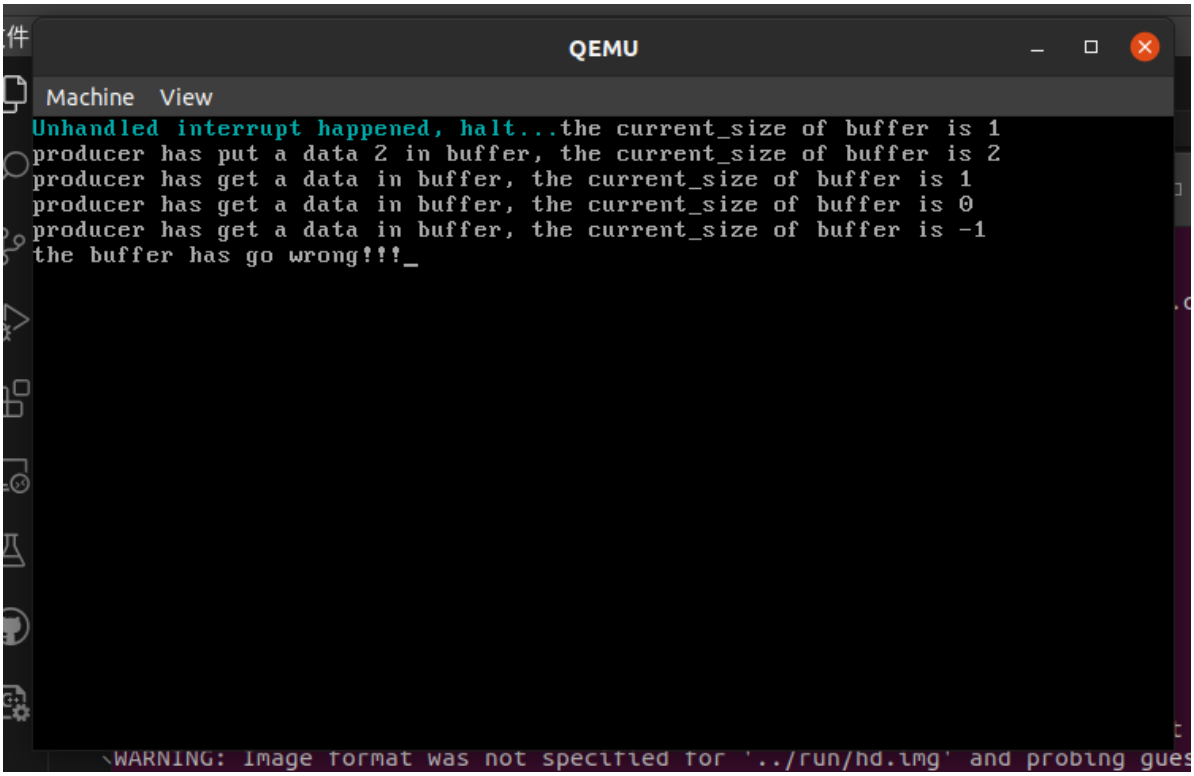
接着我们定义几个线程来模拟消费者于生产者，通过延迟时间的不同保证消费者快于生产者耗光资源，同时继续访问缓冲区，导致错误：

```
void producer1(void* arg) {
    while(true) {
        // semaphore.producer_P();
        b.put_data('1');
        // semaphore.producer_V();
        int c = 1000000000;
        while(c--);
    }
}

void producer2(void* arg) {
    while(true) {
        // semaphore.producer_P();
        b.put_data('2');
        // semaphore.producer_V();
        int c = 1000000000;
        while(c--);
    }
}
// void producer3(void* arg) {
//     while(true) {
//         b.put_data('3');
//         int c = 1000000000;
//         while(c--);
//     }
// }
void consumer1(void* arg) {
    while(true) {
        // semaphore.consumer_P();
        b.get_data();
        // semaphore.consumer_V();
        int c = 100000000;
        while(c--);
    }
}
```

```
void first_thread_PC_problem(void* arg) {
    stdio.moveCursor(0);
    semaphore.initialize(BUFFER_SIZE);
    for (int i = 0; i < 25 * 80; ++i)
    {
        stdio.print(' ');
    }
    stdio.moveCursor(0);
    programManager.executeThread(producer1, nullptr, "producer1", 1);
    programManager.executeThread(producer2, nullptr, "producer2", 1);
    programManager.executeThread(consumer1, nullptr, "consumer1", 1);
    // programManager.executeThread(producer3, nullptr, "producer3", 1);
    asm_halt();
}
```

```
// 创建第一个线程
int pid = programManager.executeThread(first_thread_PC_problem, nullptr, "first thread", 1);
if (pid == -1)
{
    printf("can not execute thread\n");
    asm_halt();
}
```



可以看到，正如预想的那样，消费者耗光资源后继续尝试消耗资源，导致错误发生。

生产者消费者问题解决：

为了用信号量解决这个问题，我们定义了一个新的锁，定义在sync.h，实现在sync.cpp中。

```
class Semaphore
{
private:
    //计数信号量
    uint32 counter;
    uint32 size;
    //生产者和消费者的挂起队列
    List waiting_p, waiting_c;
    //自旋锁保互斥
    SpinLock semLock;

public:
    Semaphore();
    void initialize(uint32 counter);
    void producer_P();
    void consumer_P();
    void producer_V();
    void consumer_V();

};
#endif
```

这里面我们定义了一个计数信号量（其与队列中的资源数量保持一致），以及一个自旋锁来保证操作互斥，同时定义了两个阻塞队列分别存放阻塞的生产者与消费者，同时定义了初始化以及对应的PV操作，具体定义如下：

初始化函数：

```
Semaphore::Semaphore()
{
    initialize(10);
}

void Semaphore::initialize(uint32 size)
{
    this -> size = size;
    this -> counter = 0;
    semLock.initialize();
    waiting_p.initialize();
    waiting_c.initialize();
}
```

生成者的P操作，首先尝试获取锁，获取锁后，判断队列中的资源数量，若未达到上限，信号量加1，因为此时锁还是关闭的，所以边界区只有它一个线程，先信号量加1是不会产生错误的，然后跳出循环，执行真正的放数据操作。

若达到上限，则阻塞进程，同时释放锁，调度下一个进程。

```
void Semaphore::producer_P()
{
    PCB *cur = nullptr;

    while (true)
    {
        semLock.lock();
        if (counter < size)
        {
            ++counter;
            // semLock.unlock();
            return;
        }

        cur = programManager.running;
        waiting_p.push_back(&(cur->tagInGeneralList));
        cur->status = ProgramStatus::BLOCKED;

        semLock.unlock();
        programManager.schedule();
    }
}
```

消费者的P操作大体一致，只不过其是对信号量进行消耗：

```
void Semaphore::consumer_P() {
    PCB *cur = nullptr;

    while (true)
    {
        semLock.lock();
        if (counter > 0)
        {
            --counter;
            // semLock.unlock();
            return;
        }

        cur = programManager.running;
        waiting_c.push_back(&(cur->tagInGeneralList));
        cur->status = ProgramStatus::BLOCKED;

        semLock.unlock();
        programManager.schedule();
    }
}
```

生产者的V操作，因为已经放入一个资源，所以**生产者在结束时要检查消费者阻塞队列是否空闲**，对消费者队列的消费者线程进行唤醒，放入就绪队列同时也要释放锁。由于依旧是采用的MESA模型所以唤醒的线程不一定会获取资源，还要进行判断，所以P操作里面是写成循环的形式。

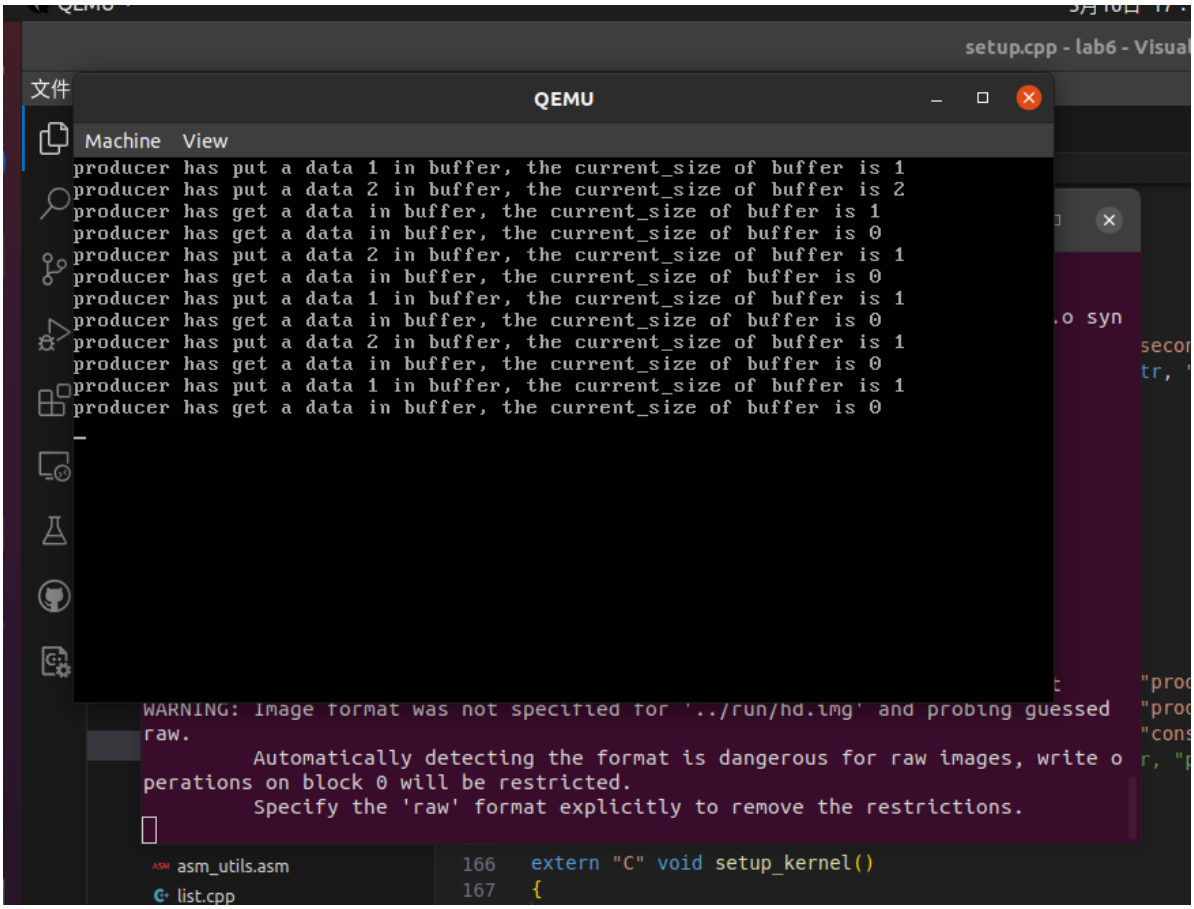
```
void Semaphore::producer_V()
{
    if (waiting_c.size())
    {
        PCB *program = ListItem2PCB(waiting_c.front(), tagInGeneralList);
        waiting_c.pop_front();
        semLock.unlock();
        programManager.MESA_WakeUp(program);
    }
    else
    {
        semLock.unlock();
    }
}
```

与生产者的V操作同理，消费者结束时也要检查生产者阻塞队列是否空闲，然后进行唤醒：

```cpp
void Semaphore::consumer_V()
{
    if (waiting_p.size())
    {
        PCB *program = ListItem2PCB(waiting_p.front(), tagInGeneralList);
        waiting_p.pop_front();
        semLock.unlock();
        programManager.MESA_WakeUp(program);
    }
    else
    {
        semLock.unlock();
    }
}
```

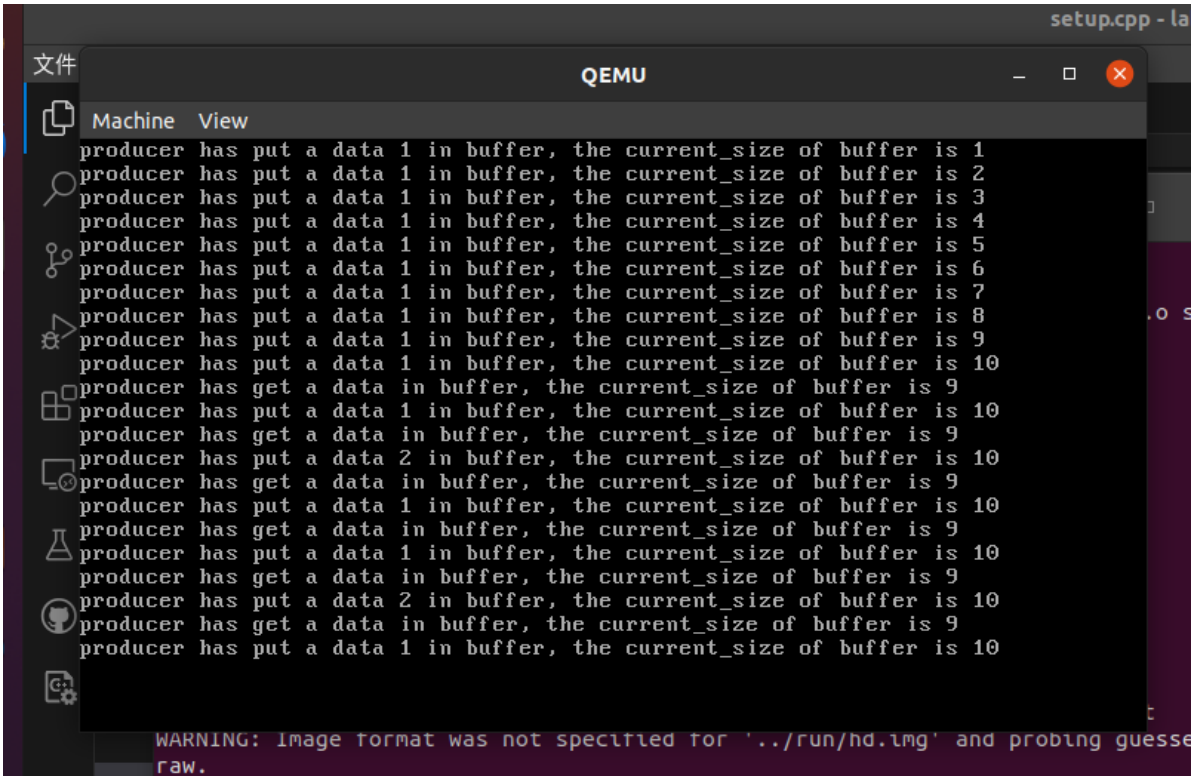
以上最关键的是消费者与生成者的唤醒时机，即在对方完成操作后进行唤醒。同时为了防止竞争，要求锁直到操作完成后才进行释放（或者线程挂起时释放）。

实例化锁后，在上面的线程中加入以上锁，（将上面线程里的注释放开），再次运行结果如下：



可以看到，当资源为空时，消费者会等待直到生产者将资源放入缓冲区之后再消费，符合预期。

将生成者生产速度加快，消费者消费速度减慢，运行程序如下，可以看到生产者在队满时会等待直到消费者进行消费后再进行数据放置，符合预期：



哲学家就餐问题：

代码位于文件夹3中。

问题复现与解决

首先编写Fork与Philosopher类，其中Fork类继承信号量类，因此其自带互斥，这个本身就已经进行了解决，因为一根筷子，所以信号量初始化为1，这里为教程中的信号量新添加了一个传参的初始化函数。里面定义了筷子id，以及尝试获取筷子与释放筷子的方法其中打印了一些信息，如下：

```
class Fork : public Semaphore {
    public:
        Fork(int id) : Semaphore(1), id(id) {

        }
        void tryFork() {
            P();
            printf("have got the fork, the fork id is %d\n",id);
        }
        void releaseFork() {
            printf("release the fork, the id is %d\n",id);
            V();
        }
        int getid() {
            return id;
        }
    private:
        int id;
};
```

下面是哲学家的类定义，里面定义了指针分别指向左右筷子，然后还有哲学家的id，（这个截图已经是解决死锁的代码了）以及尝试获取筷子的操作eat还有思考操作think。

```
class Philosopher {
    public:
        Philosopher(Fork* left, Fork* right, int id) : leftfork(left), rightfork(right), id(id) {

        }
        void think() {
            int count = 0;
            while(count) {
                count--;
            }
        }
        void eat() {
            if(id % 2 == 0) {
                printf("Philosopher id %d, try to get the fork whose id is %d\n",id, leftfork -> getid());
                leftfork -> tryFork();
                int delay = 0xfffffff;
                while(delay) {
                    delay--;
                }
                printf("Philosopher id %d, try to get the fork whose id is %d\n",id, rightfork -> getid());
                rightfork -> tryFork();
                printf("Philosopher id %d, is eating!!!!\n",id);
                int count = 100000000;
                while(count) {
                    count--;
                }
                printf("Philosopher id %d, release fork:\n",id);
                rightfork -> releaseFork();
                leftfork -> releaseFork();
            } else {
                printf("Philosopher id %d, try to get the fork whose id is %d\n",id, rightfork -> getid());
                rightfork -> tryFork();
                int delay = 0xfffffff;
                while(delay) {
                    delay--;
                }
                printf("Philosopher id %d, try to get the fork whose id is %d\n",id, leftfork -> getid());
                leftfork -> tryFork();
                printf("Philosopher id %d, is eating!!!!\n",id);
                int count = 100000000;
                while(count) {
                    count--;
                }
                printf("Philosopher id %d, release fork:\n",id);
                leftfork -> releaseFork();
                rightfork -> releaseFork();
            }

        }
        int getid() {
            return id;
        }
    private:
        int id;
        Fork* leftfork;
        Fork* rightfork;
};
```

设计第一个线程如下，在里面初始化已经创建的Fork以及Philosopher实例，里面创建子线程，它们共用一个线程函数,线程函数里面调用类中的方法进行模拟，如下：

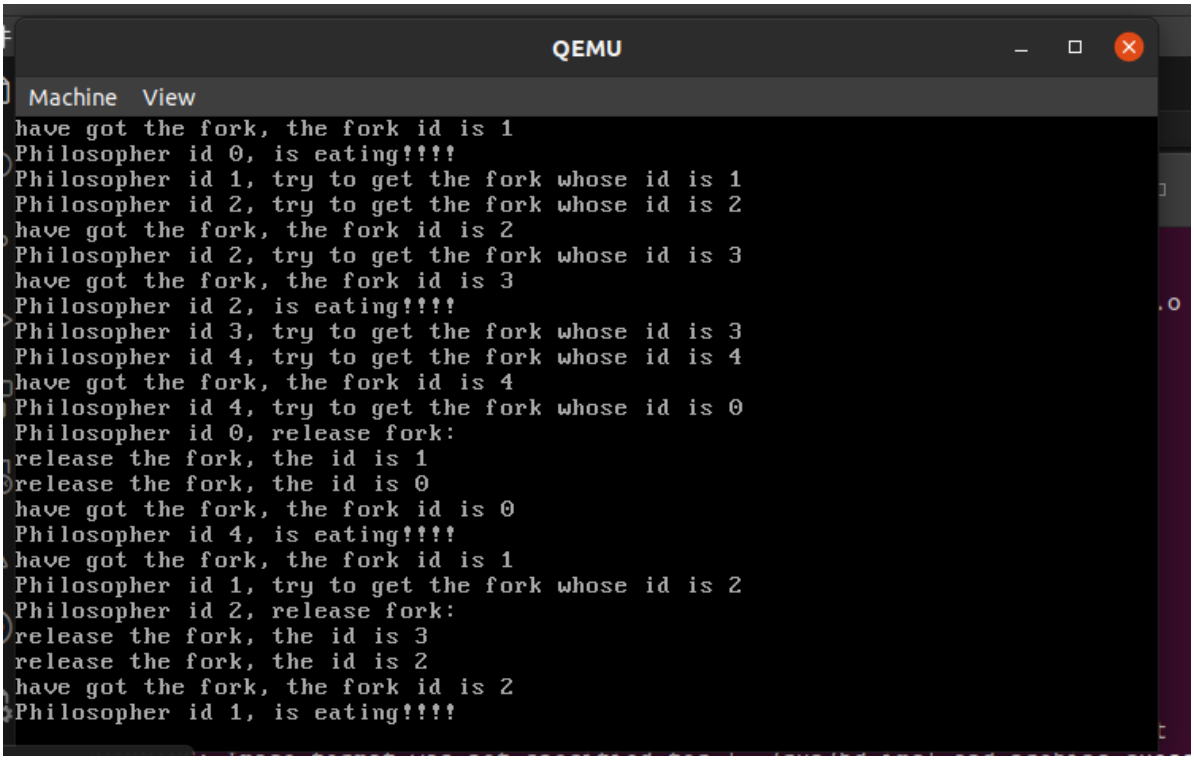
```
void first_thread_DPP(void *arg)
{
    // 5支筷子
    Fork forks[PHILOSOPHER_NUM] = {Fork(0), Fork(1), Fork(2), Fork(3), Fork(4)};
    Philosopher philosophers[PHILOSOPHER_NUM] = {Philosopher(&forks[0], &forks[1], 0), Philosopher(&forks[1], &forks[2], 1), Philosophe
    // 第1个线程不可以返回
    stdio.moveCursor(0);
    for (int i = 0; i < 25 * 80; ++i)
    {
        stdio.print(' ');
    }
    stdio.moveCursor(0);
    // for(int i = 0; i < PHILOSOPHER_NUM; i++) {
    //     printf("forks[%d]\n", forks[i].getid());
    // }
    // for(int i = 0; i < PHILOSOPHER_NUM; i++) {
    //     printf("forks[%d]\n", philosophers[i].getid());
    // }
    for(int i = 0; i < PHILOSOPHER_NUM; i++) {
        // philosophers[i] = new Philosopher(forks[i], forks[(i + 1) % PHILOSOPHER_NUM], i);
        char str[13] = "philosopher";
        str[11] = i + '0';
        str[12] = '\0';
        programManager.executeThread(thread_philosopher, &philosophers[i], str, 1);
    }

    // programManager.executeThread(a_mother, nullptr, "second thread", 1);

    asm_halt();
}
```

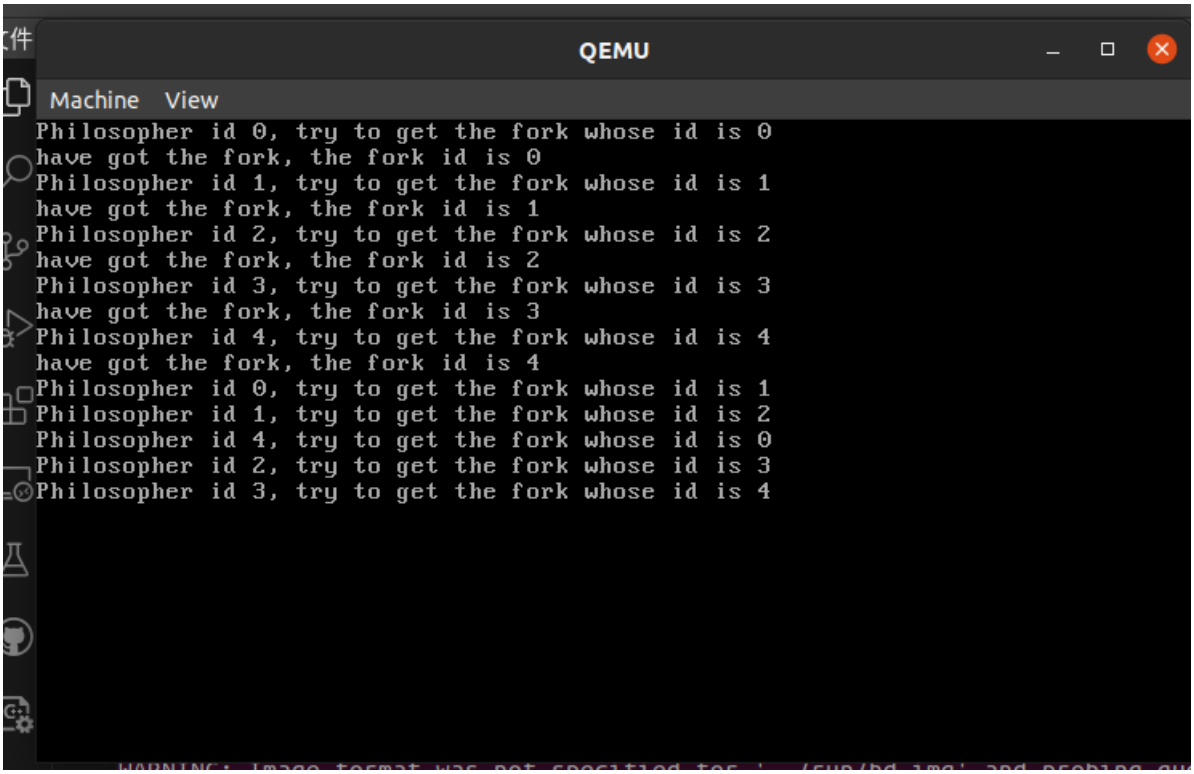
```
void thread_philosopher(void* arg) {
    while(true) {
        ((Philosopher*)arg) -> think();
        ((Philosopher*)arg) -> eat();
         int count = 100000000;
         while(count) {
             count--;
         }
    }
}
```

执行，结果如下（在未解决死锁之前的截图，以上代码都是解决完死锁后的，区别是没有哲学家奇偶判断以及每个筷子之间的延迟）：



解决死锁问题：

在eat函数中，两根筷子获取中间加入延迟，（代码在上面的图片中）即每个哲学家只拿起其中一根筷子，还来不及拿起另一根筷子就被调度下去，这样所有哲学家都拿一根筷子，等待另一根，产生死锁：



解决方法是规定id为奇数的哲学家首先尝试拿起右边的筷子，偶数的哲学家首先尝试拿起左边的筷子，这样就会相邻的哲学家竞争同一支筷子（代码在上面的图片中），种交替的筷子拿取方式确保了每位哲学家最多只需要等待一个筷子，避免了循环等待，从而避免了死锁的发生：